

Шпаргалка по коду: Реализация алгоритмов Machine Learning

23 июня 2026 г.

Основные этапы решения задачи машинного обучения:

1. Постановка задачи;
2. Выделение признаков;
3. Формирование выборки;
4. Выбор функционала ошибки;
5. Предобработка данных;
6. Построение модели;
7. Оценивание качества модели.

Содержание

1 Решающие деревья: Классификация и Регрессия	2
1.1 Импорт библиотек и базовый пайплайн обучения	2
1.2 Гиперпараметры регуляризации и их влияние на оверфиттинг	2
1.3 Интерпретация: Визуализация структуры графа и Важность признаков	3
1.4 Построение разделяющей поверхности (Decision Boundary)	3
2 Разложение ошибки (Bias-Variance) и Ансамбли моделей	4
2.1 Математическое разложение среднеквадратичной ошибки (MSE)	4
2.2 Бэггинг (Bagging) и Out-of-Bag (OOB) валидация	4
2.3 Случайный лес (Random Forest)	5
2.4 Введение в Бустинг (Boosting)	5
3 Градиентный бустинг и современные библиотеки (XGBoost, LightGBM, CatBoost)	6
3.1 Классический градиентный бустинг (Base GBM)	6
3.2 XGBoost (Extreme Gradient Boosting)	6
3.3 LightGBM (Light Gradient Boosting Machine)	7
3.4 CatBoost (Categorical Boosting)	7
4 Обучение без учителя: Методы кластеризации	8
4.1 Алгоритм K-Means (K-средних) и метод локтя	8
4.2 Плотностной алгоритм DBSCAN	8
4.3 Иерархическая кластеризация (Agglomerative Clustering)	9
4.4 Метрики оценки качества кластеризации	9
5 Отбор признаков и снижение размерности	10
5.1 Методы отбора признаков (Feature Selection)	10
5.2 Метод главных компонент (PCA)	10
5.3 Нелинейное снижение размерности: t-SNE	11

1 Решающие деревья: Классификация и Регрессия

1.1 Импорт библиотек и базовый пайплайн обучения

Для построения решающих деревьев в библиотеке `scikit-learn` используются классы `DecisionTreeClassifier` (для задач классификации) и `DecisionTreeRegressor` (для задач регрессии).

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.tree import DecisionTreeClassifier, DecisionTreeRegressor, plot_tree
4 from sklearn.datasets import load_iris
5 from sklearn.model_selection import train_test_split
6 from sklearn.metrics import accuracy_score
7
8 # 1. Загрузка данных (на примере датасета Iris)
9 data = load_iris()
10 X = data.data[:, 2:] # Берем длину и ширину лепестка для наглядности
11 y = data.target
12
13 # 2. Инициализация и обучение дерева классификации
14 # Обязательно фиксируем max_depth и random_state для воспроизводимости
15 tree_clf = DecisionTreeClassifier(max_depth=3, criterion='gini', random_state=42)
16 tree_clf.fit(X, y)
17
18 # 3. Предсказание меток классов и вероятностей принадлежности к классам
19 y_pred = tree_clf.predict(X)
20 y_proba = tree_clf.predict_proba(X) # Выдает вектор долей классов в целевом листе
21
22 # 4. Инициализация и обучение дерева регрессии
23 tree_reg = DecisionTreeRegressor(max_depth=4, criterion='squared_error', random_state=42)
24 tree_reg.fit(X, y)
25 y_reg_pred = tree_reg.predict(X) # Выдает константное среднее значение листа
```

Листинг 1: Обучение модели и получение предсказаний

1.2 Гиперпараметры регуляризации и их влияние на оверфиттинг

Без настройки параметров дерево способно разрастаться до тех пор, пока в каждом листе не окажется по одному объекту, что ведет к катастрофическому переобучению (низкое смещение, огромный разброс).

- **criterion** (string, по умолчанию='gini' / 'squared_error'):
 - **Классификация:** 'gini' (индекс нечистоты Джини) или 'entropy' (энтропия Шеннона). Джини вычисляется быстрее, энтропия сильнее штрафует смешанные узлы.
 - **Регрессия:** 'squared_error' (MSE, среднее), 'absolute_error' (MAE, медиана), 'friedman_mse' или 'poisson'. MAE устойчив к выбросам.
- **max_depth** (int, по умолчанию=None): Максимально допустимая глубина дерева.
 - **Плюсы:** Самый мощный и понятный инструмент контроля переобучения.
 - **Минусы:** Слишком маленькое значение (1-2) приводит к сильному недообучению (высокий bias).
- **min_samples_split** (int или float, по умолчанию=2): Минимальное число объектов в узле, при котором система разрешит сделать его дальнейшее ветвление.
- **min_samples_leaf** (int или float, по умолчанию=1): Минимальное количество объектов, которое обязано остаться в получившихся дочерних листьях после сплита.
 - **Плюсы:** Идеально отсекает изолированные шумовые точки и мелкие аномалии, сглаживая прогноз. Рекомендуется ставить значение от 5 и выше на реальных шумных данных.
- **max_features** (int, float или string, по умолчанию=None): Число признаков, которые случайно выбираются моделью для поиска оптимального сплита в каждом узле. Помогает бороться с коррелированностью (база для случайного леса).

1.3 Интерпретация: Визуализация структуры графа и Важность признаков

Деревья хороши своей «прозрачностью». Мы можем извлечь точные пороговые правила принятия решений и узнать вклад каждой переменной в финальный результат.

```
1 # 1. Построение графического графа правил (plot_tree)
2 plt.figure(figsize=(14, 8))
3 plot_tree(
4     tree_clf,
5     feature_names=["petal length", "petal width"], # Подписи признаков
6     class_names=data.target_names,               # Подписи классов
7     filled=True,                                  # Цветовая заливка узлов по доминирующему
8     rounded=True,                                # Красивые скругленные углы у блоков
9     fontsize=10
10 )
11 plt.show()
12
13 # 2. Извлечение важности признаков (Feature Importances)
14 # Важность считается как суммарное падение критерия информативности (Gini/MSE) во всех узлах,
15 # где был сплит по данному признаку
16 importances = tree_clf.feature_importances_
17
18 for feature_name, importance_value in zip(["petal length", "petal width"], importances):
19     print(f"Признак: {feature_name} -> Относительная важность: {importance_value:.4f}")
```

Листинг 2: Отрисовка структуры дерева и Feature Importances

1.4 Построение разделяющей поверхности (Decision Boundary)

Код для генерации координатной сетки и отображения ступенчатых ортогональных границ, которые дерево выстраивает в пространстве признаков.

```
1 # 1. Создаем плотную сетку точек по диапазону признаков выборки
2 x_min, x_max = X[:, 0].min() - 1.0, X[:, 0].max() + 1.0
3 y_min, y_max = X[:, 1].min() - 1.0, X[:, 1].max() + 1.0
4 xx, yy = np.meshgrid(
5     np.linspace(x_min, x_max, 500),
6     np.linspace(y_min, y_max, 500)
7 )
8
9 # 2. Получаем предсказание класса для каждой точки координатной сетки
10 grid_points = np.c_[xx.ravel(), yy.ravel()]
11 z = tree_clf.predict(grid_points).reshape(xx.shape)
12
13 # 3. Отрисовка фоновых цветовых зон классов
14 plt.figure(figsize=(10, 6))
15 plt.contourf(xx, yy, z, alpha=0.3, cmap=plt.cm.autumn)
16
17 # 4. Наложение реальных точек объектов поверх карты предсказаний
18 colors = ['#FF0000', '#FF7700', '#FFFF00']
19 for i, color_code in zip(range(3), colors):
20     plt.scatter(
21         X[y == i, 0], X[y == i, 1],
22         c=color_code, label=data.target_names[i],
23         edgecolor='k'
24     )
25
26 plt.xlabel("Длина лепестка, см")
27 plt.ylabel("Ширина лепестка, см")
28 plt.legend(fontsize=12)
29
30 # Очистка графика: убираем лишние верхнюю и правую рамки (фишка Seaborn/Matplotlib style)
31 for ax in plt.gcf().get_axes():
32     ax.spines['top'].set_visible(False)
33     ax.spines['right'].set_visible(False)
34
35 plt.show()
```

Листинг 3: Генерация сетки и отрисовка карты классов

2 Разложение ошибки (Bias-Variance) и Ансамбли моделей

2.1 Математическое разложение среднеквадратичной ошибки (MSE)

На лекции и семинаре выведена формула представления среднеквадратичной ошибки алгоритма регрессии $\mu(X)$, обученного на случайной выборке X , в виде трёх компонент:

$$L(\mu) = \mathbb{E}_{x,y} [\mathbb{E}_X [(y - \mu(X)(x))^2]] = \text{Шум} + \text{Смещение}^2 + \text{Разброс}$$

$$\text{Шум} = \mathbb{E}_{x,y} [(y - \mathbb{E}[yx])^2]$$

Характеризует неустранимую ошибку, обученную природой данных и случайными флуктуациями целевой переменной. Не зависит от выбора модели.

$$\text{Смещение}^2 = \mathbb{E}_x [(\mathbb{E}_X [\mu(X)(x)] - \mathbb{E}[yx])^2]$$

Показывает отклонение среднего ответа обученного алгоритма от истинного матожидания целевой переменной. Высокое смещение — признак недообучения (модель слишком простая).

$$\text{Разброс} = \mathbb{E}_x [\mathbb{E}_X [(\mu(X)(x) - \mathbb{E}_X [\mu(X)(x)])^2]]$$

Мера чувствительности алгоритма к изменению обучающей выборки X . Высокий разброс — признак переобучения (модель подстраивается под шум).

2.2 Бэггинг (Bagging) и Out-of-Bag (OOB) валидация

Бэггинг (Bootstrap Aggregating) обучает множество базовых моделей параллельно на случайных подвыборках с возвращением.

- **Влияние на ошибку:** Снижает **разброс** композиции за счет усреднения, оставляя смещение практически неизменным. Идеален для базовых моделей с низким смещением и высоким разбросом (глубокие деревья).
- **Out-of-Bag оценка:** При генерации бутстрап-выборки размера ℓ в нее в среднем не попадает около 37% объектов: $\lim_{\ell \rightarrow \infty} (1 - \frac{1}{\ell})^\ell = \frac{1}{e} \approx 0.368$. Их используют для оценки обобщающей способности без выделения отдельного валидационного сета.

```
1 from sklearn.ensemble import BaggingClassifier
2 from sklearn.tree import DecisionTreeClassifier
3 from sklearn.datasets import make_classification
4
5 # 1. Генерация синтетических данных
6 X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
7
8 # 2. Инициализация бэггинга над глубокими переобученными деревьями
9 # oob_score=True автоматически считает качество на несэмплированных данных
10 bag_clf = BaggingClassifier(
11     estimator=DecisionTreeClassifier(max_depth=None, random_state=42),
12     n_estimators=100,           # Количество параллельных базовых моделей
13     max_samples=1.0,           # Доля объектов выборки для одного бутстрапа
14     max_features=1.0,          # Доля признаков для одной базовой модели
15     bootstrap=True,            # Использовать ли сэмплирование с возвращением
16     oob_score=True,            # Включение Out-of-Bag валидации
17     n_jobs=-1,                 # Распараллеливание на все ядра процессора
18     random_state=42
19 )
20
21 # 3. Обучение и извлечение OOB метрики качества
22 bag_clf.fit(X, y)
23 print(f"Out-of-Bag Accuracy: {bag_clf.oob_score_:.4f}")
```

Листинг 4: Реализация бэггинга и оценка Out-of-Bag в Sklearn

2.3 Случайный лес (Random Forest)

Случайный лес улучшает классический бэггинг над решающими деревьями, принудительно **декоррелируя** их между собой. В каждом узле каждого дерева оптимальный сплит ищется не среди всех признаков, а среди случайного подмножества размера m .

- **Эвристика выбора m :** Для классификации обычно $m = \sqrt{d}$, для регрессии $m = \frac{d}{3}$ (где d — общее число признаков).
- **Свойства:** Не переобучается при увеличении числа деревьев (`n_estimators`), встроенно оценивает важность признаков.

```
1 from sklearn.ensemble import RandomForestClassifier
2 from sklearn.metrics import classification_report
3
4 # 1. Инициализация и обучение модели RandomForest
5 rf_clf = RandomForestClassifier(
6     n_estimators=150,
7     criterion='gini',
8     max_depth=8,
9     min_samples_leaf=4,
10    max_features='sqrt',    # Тот самый отбор подмножества признаков (m = sqrt(d))
11    bootstrap=True,
12    random_state=42,
13    n_jobs=-1
14 )
15 rf_clf.fit(X, y)
16
17 # 2. Быстрое извлечение оценок важности признаков (MDI Importance)
18 # Считается по усредненному падению критерия Джини/MSE на узлах сплитов
19 feature_gains = rf_clf.feature_importances_
20 print(f"Важность первых трех признаков: {feature_gains[:3]}")
```

Листинг 5: Случайный лес: классификация и тюнинг

2.4 Введение в Бустинг (Boosting)

В отличие от бэггинга, бустинг — это строго **последовательный** метод построения ансамбля, где каждый следующий алгоритм обучается исправлять ошибки (остатки), допущенные всей текущей композицией его предшественников.

- **Влияние на ошибку:** Пошагово уменьшает **смещение (bias)** композиции.
- **Базовые модели:** В качестве базы выбираются алгоритмы с высоким смещением и маленьким разбросом (например, неглубокие решающие деревья — «пеньки» с `max_depth=2` или `max_depth=3`).

```
1 from sklearn.ensemble import GradientBoostingClassifier
2
3 # Инициализация градиентного бустинга (понижает bias)
4 gb_clf = GradientBoostingClassifier(
5     loss='log_loss',    # Оптимизируемая суррогатная функция потерь
6     learning_rate=0.1,  # Шаг градиентного спуска (темп обучения композиции)
7     n_estimators=100,   # Число последовательно строящихся деревьев
8     max_depth=3,        # Фиксируем маленькую глубину базовых деревьев
9     subsample=0.8,      # Стохастический бустинг (обучение на 80% объектов)
10    random_state=42
11 )
12 gb_clf.fit(X, y)
```

Листинг 6: Базовый градиентный бустинг в Sklearn

3 Градиентный бустинг и современные библиотеки (XGBoost, LightGBM, CatBoost)

3.1 Классический градиентный бустинг (Base GBM)

Градиентный бустинг строит композицию в виде взвешенной суммы базовых алгоритмов: $a_N(x) = \sum_{n=0}^N \gamma_n b_n(x)$. Обучение происходит строго последовательно. На каждом шаге n минимизируется функционал ошибки относительно нового базового алгоритма $b_n(x)$ и его веса γ_n .

Для произвольной дифференцируемой функции потерь $L(y, z)$ задача оптимизации аппроксимируется сдвигом прогнозов по направлению антиградиента. Вычисляются векторы псевдоостатков (направления спуска в пространстве ответов):

$$s_i = - \left[\frac{\partial L(y_i, z)}{\partial z} \right]_{z=a_{n-1}(x_i)}$$

Новый базовый алгоритм $b_n(x)$ обучается предсказывать псевдоостатки s_i путем минимизации среднеквадратичной ошибки (MSE): $\sum_{i=1}^{\ell} (s_i - b(x_i))^2$. После фиксации структуры $b_n(x)$ находят оптимальный одномерный шаг $\gamma_n = \arg \min_{\gamma} \sum_{i=1}^{\ell} L(y_i, a_{n-1}(x_i) + \gamma b_n(x_i))$. В случае решающих деревьев константный сдвиг оптимизируется независимо для каждого листа.

```
1 from sklearn.ensemble import GradientBoostingClassifier
2 from sklearn.datasets import make_classification
3
4 X, y = make_classification(n_samples=1000, n_features=20, random_state=42)
5
6 # Базовый бустинг последовательно минимизирует смещение (bias)
7 gb_clf = GradientBoostingClassifier(
8     loss='log_loss',           # Оптимизируемая функция потерь
9     learning_rate=0.1,         # Шаг градиентного спуска (темп обучения композиции)
10    n_estimators=100,          # Количество последовательных деревьев
11    max_depth=3,               # Малая глубина базовых деревьев для контроля разброса
12    subsample=0.8,             # Стохастический бустинг (выборка 80 процентов строк)
13    random_state=42
14 )
15 gb_clf.fit(X, y)
```

Листинг 7: Классический бустинг в Sklearn

3.2 XGBoost (Extreme Gradient Boosting)

XGBoost использует разложение Тейлора второго порядка для аппроксимации функции потерь, вычисляя градиенты $g_i = \frac{\partial L(y_i, a_{n-1}(x_i))}{\partial a_{n-1}(x_i)}$ и вторые производные (гессисаны) $h_i = \frac{\partial^2 L(y_i, a_{n-1}(x_i))}{\partial a_{n-1}(x_i)^2}$.

Функционал ошибки на шаге n регуляризуется штрафом за структуру дерева (число листьев T) и L2-норму весов в листьях w_j :

$$\tilde{\mathcal{L}}^{(n)} \approx \sum_{j=1}^T \left[\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i + \lambda \right) w_j^2 \right] + \gamma T$$

- **Оптимальные веса в листьях:** Дифференцирование по w_j дает аналитическую формулу: $w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda}$.
- **Критерий ветвления (Gain):** Качество сплита узла I на подмножества I_L и I_R оценивается по приросту структуры:

$$\text{Gain} = \frac{1}{2} \left[\frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma$$

```
1 from xgboost import XGBClassifier
2
3 # Мощная библиотека со встроенной L2-регуляризацией листьев
4 xgb_clf = XGBClassifier(
5     n_estimators=100,
6     max_depth=5,
7     learning_rate=0.05,
8     reg_lambda=1.0,           # Коэффициент Лямбда для L2-регуляризации весов
9     gamma=0.1,               # Минимальный Gain для совершения нового сплита
10    tree_method='hist',       # Гистограммный метод для ускорения вычислений
11    random_state=42
12 )
```

3.3 LightGBM (Light Gradient Boosting Machine)

Разработан Microsoft для экстремального ускорения вычислений и снижения потребления оперативной памяти за счет следующих ключевых алгоритмических трюков:

- **Leaf-wise пост дерева:** Дерево растет не по уровням (level-wise), а путем расщепления того конкретного листа, который дает максимальное уменьшение ошибки. Позволяет строить глубокие асимметричные деревья, но требует строгой регуляризации гиперпараметрами `max_depth` и `min_data_in_leaf` для исключения переобучения.
- **Histogram-based алгоритм:** Непрерывные признаки квантуются в дискретные корзины (bins). Поиск сплита ускоряется с $O(\ell)$ до $O(\text{nbins})$.
- **GOSS (Gradient-based One-Side Sampling):** Объекты с большими градиентами (высокой ошибкой) сохраняются полностью, а объекты с маленькими градиентами сэмплятся случайно. Снижает объем обрабатываемых строк выборки.
- **EFB (Exclusive Feature Bundling):** Безопасно объединяет взаимоисключающие разреженные признаки (например, после One-Hot кодирования) в единые бандлы, сокращая число столбцов d .

```
1 from lightgbm import LGBMClassifier
2
3 lgb_clf = LGBMClassifier(
4     n_estimators=120,
5     learning_rate=0.08,
6     num_leaves=31,           # Главный параметр сложности при Leaf-wise росте
7     max_depth=6,            # Ограничение глубины во избежание оверфиттинга
8     min_child_samples=20,    # Минимальное число объектов в листе
9     boosting_type='gbdt',    # Режимы: gbdт (дефолт) или goss (One-Side Sampling)
10    random_state=42
11 )
12 lgb_clf.fit(X, y)
```

Листинг 9: Настройка LightGBM

3.4 CatBoost (Categorical Boosting)

Разработан Яндексом. Сфокусирован на идеальной «из коробки» работе с категориальными данными и подавлении динамического переобучения композиции.

- **Ordered Boosting:** Обычный бустинг страдает от утечки таргета (Target Leakage), так как остатки на объекте x_i считаются по модели, которая уже неявно видела его значение y_i . CatBoost обучает вспомогательные модели на случайных перестановках выборки, считая остатки для объекта только по элементам, идущим строго перед ним.
- **Симметричные (Oblivious) деревья:** На каждом уровне дерева используется строго один и тот же предикат (индекс признака и порог) для всех узлов текущей глубины. Структура сбалансирована, защищена от переобучения, а инференс выполняется мгновенно через быстрые битовые маски (обращение к памяти как к хэш-таблице).
- **Продвинутый Target Encoding:** Встроенный расчет счетчиков по категориям сглаживается априорной вероятностью и считается в онлайн-режиме по случайным перестановкам.

```
1 from catboost import CatBoostClassifier
2
3 # Допустим, признаки с индексами 0 и 3 являются категориальными (строки или int)
4 cat_features_indices = [0, 3]
5
6 cat_clf = CatBoostClassifier(
7     iterations=150,          # Количество деревьев (аналог n_estimators)
8     learning_rate=0.1,
9     depth=6,                 # Глубина симметричного дерева
10    l2_leaf_reg=3.0,          # Коэффициент L2-регуляризации весов в листьях
11    verbose=False,           # Отключение вывода логов итераций обучения
12    random_seed=42
13 )
14 # Передаем индексы категориальных колонок напрямую в метод fit
15 # cat_clf.fit(X_train, y_train, cat_features=cat_features_indices)
```

Листинг 10: Реализация CatBoost с категориальными фичами

4 Обучение без учителя: Методы кластеризации

4.1 Алгоритм K-Means (К-средних) и метод локтя

Минимизирует суммарное квадратичное отклонение точек от центроидов своих кластеров (Inertia). Алгоритм чувствителен к начальной инициализации центроидов и масштабу признаков (требует предварительного применения StandardScaler).

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.cluster import KMeans
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.datasets import make_blobs
6
7 # 1. Генерация тестовых данных и масштабирование
8 X, y = make_blobs(n_samples=500, centers=4, cluster_std=0.60, random_state=42)
9 scaler = StandardScaler()
10 X_scaled = scaler.fit_transform(X)
11
12 # 2. Обучение модели K-Means
13 # Парам. init='k-means++' выбирает начальные центры на удалении друг от друга
14 kmeans = KMeans(n_clusters=4, init='k-means++', n_init=10, max_iter=300, random_state=42)
15 kmeans.fit(X_scaled)
16
17 # 3. Извлечение результатов работы алгоритма
18 cluster_labels = kmeans.labels_ # Массив индексов кластеров для каждого объекта
19 centroids = kmeans.cluster_centers_ # Координаты финальных центров кластеров
20 inertia_value = kmeans.inertia_ # Сумма квадратов расстояний до центроидов (WSSR)
21
22 # 4. Реализация метода локтя (Elbow Method) для выбора оптимального K
23 inertia_list = []
24 k_range = range(1, 11)
25 for k in k_range:
26     km = KMeans(n_clusters=k, init='k-means++', n_init=5, random_state=42)
27     km.fit(X_scaled)
28     inertia_list.append(km.inertia_)
```

Листинг 11: Кластеризация K-Means и вычисление инерции

- `n_clusters` (int, по умолчанию=8): Количество кластеров, на которое нужно разделить выборку.
- `kmeans.inertia_`: Чем меньше значение, тем плотнее объекты прижаты к центроидам. Точка перегиба («локоть») на графике зависимости `inertia_` от `K` указывает на оптимальное число кластеров.
- **Ограничение метода:** Ищет исключительно сферические или выпуклые кластеры одинакового масштаба; не справляется с вытянутыми или сложными геометрическими структурами.

4.2 Плоткостной алгоритм DBSCAN

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) выделяет кластеры как области высокой плотности точек, разделенные областями низкой плотности. Автоматически определяет количество кластеров и изолирует выбросы.

```
1 from sklearn.cluster import DBSCAN
2
3 # eps - радиус окрестности точки, min_samples - порог количества соседей
4 dbscan = DBSCAN(eps=0.3, min_samples=5, metric='euclidean')
5 dbscan.fit(X_scaled)
6
7 # Извлечение меток. Шумовые объекты (выбросы) встроено получают метку -1
8 db_labels = dbscan.labels_
9
10 # Подсчет количества выделенных кластеров (исключая шумовую категорию -1)
11 n_clusters_ = len(set(db_labels)) - (1 if -1 in db_labels else 0)
12 print(f"Количество найденных кластеров: {n_clusters_}")
13
14 # Выделение индексов стержневых (core) объектов, сформировавших кластеры
15 core_samples_mask = np.zeros_like(db_labels, dtype=bool)
16 core_samples_mask[dbscan.core_sample_indices_] = True
```

Листинг 12: Кластеризация DBSCAN и работа с шумом

- **eps (float):** Максимальное расстояние между двумя объектами, при котором они считаются соседями. Слишком маленький радиус дробит данные, слишком большой — сливает кластеры в один.
- **min_samples (int):** Минимальное число соседей (включая саму точку) в радиусе **eps**, необходимое для присвоения точке статуса основного (core) объекта.
- **Типы точек:** *Основные* (внутри плотной зоны), *Граничные* (внутри радиуса основной точки, но сами не имеют нужного числа соседей), *Шум* (метка -1, не попали ни в один радиус).
- **Плюсы:** Не требует задания числа кластеров заранее, находит кластеры любой сложной формы (дуги, кольца).
- **Минусы:** Плохо работает, если кластеры в выборке обладают существенно разной плотностью.

4.3 Иерархическая кластеризация (Agglomerative Clustering)

Строит дерево кластеров (дендрограмму) путем последовательного попарного объединения ближайших объектов или мелких групп (агломеративный подход).

```
1 from sklearn.cluster import AgglomerativeClustering
2 from scipy.cluster.hierarchy import dendrogram, linkage
3
4 # 1. Обучение модели иерархической кластеризации с фиксацией числа кластеров
5 agg_cluster = AgglomerativeClustering(n_clusters=4, metric='euclidean', linkage='ward')
6 agg_labels = agg_cluster.fit_predict(X_scaled)
7
8 # 2. Построение матрицы связей через SciPy для полной визуализации дерева
9 # Метод ward минимизирует прирост дисперсии внутри объединяемых кластеров
10 Z = linkage(X_scaled, method='ward')
11
12 # 3. Отрисовка дендрограммы
13 plt.figure(figsize=(10, 5))
14 dendrogram(Z, truncate_mode='lastp', p=12, show_leaf_counts=True)
15 plt.title("Иерархическая дендрограмма связей")
16 plt.show()
```

Листинг 13: Построение дендрограммы в Scipy и кластеризация в Sklearn

- **linkage (string):** Способ измерения расстояния между группами точек при объединении.
 - 'ward': Минимизирует сумму квадратов отклонений внутри кластеров. Ищет компактные сферы.
 - 'complete': Расстояние между самыми дальними точками двух кластеров. Защищает от шума.
 - 'average': Среднее расстояние между всеми парами точек двух кластеров.
 - 'single': Расстояние между ближайшими точками двух кластеров (порождает ленточные кластеры).

4.4 Метрики оценки качества кластеризации

Поскольку истинные метки ответов в обучении без учителя обычно отсутствуют, качество оценивают по внутренним геометрическим свойствам разбиения.

```
1 from sklearn.metrics import silhouette_score, calinski_harabasz_score, davies_bouldin_score
2 from sklearn.metrics import adjusted_rand_score
3
4 # 1. Внутренние метрики (не требуют наличия истинных ответов y)
5 sil_score = silhouette_score(X_scaled, cluster_labels) # Идеал = 1, худший = -1
6 ch_score = calinski_harabasz_score(X_scaled, cluster_labels) # Чем выше, тем лучше
7 db_score = davies_bouldin_score(X_scaled, cluster_labels) # Чем ниже, тем лучше (идеал = 0)
8
9 # 2. Внешняя метрика (применяется только если истинная разметка y известна для теста)
10 # Adjusted Rand Index корректируется на случайное угадывание. Диапазон: [-1, 1], идеал = 1
11 ari_score = adjusted_rand_score(y, cluster_labels)
```

Листинг 14: Вычисление метрик качества кластеризации

- **Силуэт (Silhouette Score):** Измеряет, насколько объект близок к своему кластеру по сравнению с чужими. Вычисляется как $s = \frac{b-a}{\max(a,b)}$, где a — среднее расстояние до точек своего кластера, b — до точек ближайшего чужого. Значение около 1 говорит о четко выраженных разделенных кластерах.

5 Отбор признаков и снижение размерности

5.1 Методы отбора признаков (Feature Selection)

Отбор признаков (Feature Selection) фильтрует исходный набор, удаляя неинформативные или избыточные фичи без изменения их исходных значений.

1. Отсечение по низкой дисперсии (Variance Threshold): Удаляет признаки, значения которых почти не меняются. Если признак бинарный и доля одного из классов равна p , его дисперсия равна $p(1 - p)$.

```
1 from sklearn.feature_selection import VarianceThreshold
2
3 # Удаляем признаки, у которых дисперсия ниже заданного порога
4 # Например, если порог 0.1, то фичи с меньшей вариативностью удалятся
5 selector = VarianceThreshold(threshold=0.1)
6 X_high_variance = selector.fit_transform(X)
```

Листинг 15: Фильтрация признаков по дисперсии

2. Одномерные статистические стратегии (SelectKBest): Оценивают силу попарной взаимосвязи между каждым отдельным признаком и целевой переменной с помощью статистических тестов.

- `f_classif`: F-значение ANOVA (для задач классификации, оценивает линейную зависимость).
- `mutual_info_classif` / `mutual_info_regression`: Взаимная информация (непараметрический метод, улавливает любые нелинейные зависимости).

```
1 from sklearn.feature_selection import SelectKBest, f_classif, mutual_info_classif
2
3 # 1. Отбор 5 лучших фич по линейному тесту ANOVA
4 anova_selector = SelectKBest(score_func=f_classif, k=5)
5 X_anova = anova_selector.fit_transform(X, y)
6
7 # 2. Отбор по нелинейному критерию взаимной информации
8 mi_selector = SelectKBest(score_func=mutual_info_classif, k=5)
9 X_mi = mi_selector.fit_transform(X, y)
10
11 # Получение посчитанных скоров для каждого признака
12 feature_scores = mi_selector.scores_
```

Листинг 16: Отбор K лучших признаков через статистические тесты

5.2 Метод главных компонент (PCA)

PCA (Principal Component Analysis) — линейный метод обучения без учителя, переводящий данные в новую ортогональную систему координат так, чтобы вдоль первой оси (первой главной компоненты) максимизировалась дисперсия данных, вдоль второй — остаток дисперсии и так далее.

- **Математическая суть:** Сводится к сингулярному разложению (SVD) центрированной матрицы признаков или спектральному разложению матрицы ковариации. Матрица весов компонент состоит из собственных векторов матрицы ковариации.
- **Обязательное требование:** Метод критически чувствителен к масштабу переменных. Перед применением PCA всегда необходимо применять `StandardScaler`.

```

1 from sklearn.decomposition import PCA
2 from sklearn.preprocessing import StandardScaler
3
4 # 1. Центрирование и масштабирование признаков
5 scaler = StandardScaler()
6 X_scaled = scaler.fit_transform(X)
7
8 # 2. Инициализация PCA с фиксацией числа итоговых компонент
9 pca = PCA(n_components=2, random_state=42)
10 X_pca = pca.fit_transform(X_scaled)
11
12 # 3. Анализ объясненной дисперсии
13 # Массив долей дисперсии, удерживаемой каждой из компонент
14 variance_ratios = pca.explained_variance_ratio_
15 print(f"Доля дисперсии первой компоненты: {variance_ratios[0]:.4f}")
16 print(f"Суммарно удержано дисперсии: {sum(variance_ratios):.4f}")
17
18 # Извлечение матрицы весов (собственных векторов) для интерпретации
19 components_matrix = pca.components_

```

Листинг 17: Снижение размерности с помощью PCA

5.3 Нелинейное снижение размерности: t-SNE

t-SNE (t-Distributed Stochastic Neighbor Embedding) — многообразное нелинейное отображение, созданное преимущественно для **визуализации** сверхмногомерных данных в 2D или 3D пространство.

- **Принцип работы:** Переводит расстояния между точками в условные вероятности сходства (в исходном пространстве используется Гауссово распределение, в низкоразмерном — распределение Стьюдента с одной степенью свободы). Минимизирует расстояние Кульбака-Лейблера (KL-divergence) между двумя распределениями.
- **Свойства модели:** Сохраняет локальные расстояния (близкие объекты остаются близкими), но может сильно искажать глобальные расстояния между далекими кластерами. Не имеет явной математической формулы трансформации, поэтому метод `transform()` отсутствует — модель обучается сразу на всей выборке через `fit_transform()`.

```

1 from sklearn.manifold import TSNE
2
3 # Перплексия (perplexity) регулирует баланс между локальной и глобальной структурой
4 # Оптимальный выбор perplexity обычно лежит в диапазоне от 5 до 50
5 tsne = TSNE(
6     n_components=2,
7     perplexity=30.0,
8     n_iter=1000,
9     learning_rate='auto',
10    init='pca',
11    random_state=42
12 )
13 X_tsne = tsne.fit_transform(X_scaled)
14
15 # Финальное значение минимизируемого расхождения Кульбака-Лейблера
16 kl_divergence_result = tsne.kl_divergence_

```

Листинг 18: Применение алгоритма t-SNE для визуализации

- **perplexity** (float, по умолчанию=30.0): Грубо говоря, определяет ожидаемое число ближайших соседей для каждой точки. Маленькие значения дробят данные на мелкие случайные группы, большие — превращают всю выборку в единое неразделяемое облако.
- **init='pca'**: Инициализация координат первыми компонентами PCA. Делает процесс оптимизации более стабильным и ускоряет сходимость градиентного спуска по сравнению со случайным стартом ('random').
- **Альтернатива (UMAP):** Более современный нелинейный метод снижения размерности (Uniform Manifold Approximation and Projection). Работает в разы быстрее t-SNE, лучше сохраняет не только локальные взаимосвязи, но и глобальную топологию данных. В отличие от t-SNE, позволяет применять обученную проекцию к новым тестовым объектам.